AGENTIC AI @ UIUC

Agentic Memory Management Lecture - 3

Understanding how AI agents store, retrieve, and leverage memory across short-term, long-term, hybrid, and user contexts

AI News: BAML & OpenClaw Acquisition

BAML Boundary AI Markup Language

What: A domain-specific language for defining typed LLM function interfaces — input/output schemas, prompt templates, and retries

Why: Eliminates brittle string parsing; type-safe structured outputs from any LLM with compile-time validation

Compiler: Rust-based compiler generates client code in Python, TypeScript, Ruby, and more

Key Feature: Schema-Aligned Parsing (SAP) — recovers valid structured data even from malformed LLM outputs

Built by: BoundaryML — a Y Combinator-backed startup focused on AI developer tooling

OpenClaw Accquired

What: Open-source AI personal agent managing email, calendar, messaging across WhatsApp, Telegram, Discord, iMessage

Scale: 1.5 million AI agents created by users as of February 2026

The Deal: OpenAI acqui-hires creator Peter Steinberger (Feb 15, 2026); code stays open-source in a foundation

Sam Altman: "We expect this will quickly become core to our product offerings" — driving next-gen personal agents

Why It Matters: Signals OpenAI's shift toward persistent personal AI agents that manage daily digital life

What We'll Cover

01 **Short-Term Memory**
Context windows, scratchpads, attention mechanisms, and token management

02 **Long-Term Memory**
Vector databases, knowledge graphs, RAG pipelines, and persistent storage

03 **Hybrid Memory Management**
Memory controllers, consolidation strategies, and eviction policies

04 **User Memory**
Personalization, preferences, interaction history, and privacy safeguards

05 **Real-World Implementations**
How ChatGPT, Claude Code, Google, and leading companies implement memory

Why Memory Matters for Agents

Without memory, agents are stateless—every interaction starts from scratch. Memory enables continuity, personalization, and learning.

4	3x	∞
Memory Types	Better Context	Scalable Recall

Short-Term Memory

Active session context & working state

Long-Term Memory

Persistent knowledge & learned patterns

Hybrid Memory

Bridging short & long-term for efficiency

User Memory

Personalized preferences & history

01

Short-Term Memory

The active working memory of an AI agent during a session

Short-Term Memory

Active context window & working state during a single session

Context Window

The current conversation and task state held in the model's input. Includes recent messages, tool outputs, and intermediate results.

Scratchpad / Buffer

Temporary notes and working variables the agent maintains mid-task. Cleared after the session or task completes.

Attention Mechanism

How the model focuses on relevant parts of short-term memory. Transformer attention prioritizes recent, salient tokens.

Token Limitations

Finite context windows (4K–200K tokens) force trade-offs. Agents must decide what to keep, compress, or discard.

STM Strategies & Techniques

01 **Sliding Window**

Keep most recent N tokens/messages, drop oldest first

02 **Summarization**

Compress older context into concise summaries to save tokens

03 **Relevance Filtering**

Score and retain only contextually relevant information

04 **Hierarchical Buffers**

Multi-level buffers: hot (recent), warm (summary), cold (archive)

Key Insight

Effective short-term memory isn't about storing everything—it's about intelligent selection and compression of the most relevant context.

02

Long-Term Memory

Persistent storage enabling knowledge retention beyond a single session

Long-Term Memory Architecture

Vector Databases

- Semantic similarity search
- Embedding-based retrieval
- Scalable to millions of entries
- e.g., Pinecone, Weaviate, Chroma

Knowledge Graphs

- Structured relationships
- Entity-based reasoning
- Traversal for multi-hop queries
- e.g., Neo4j, Amazon Neptune

Document Stores

- Full-text search
- Metadata filtering
- Hybrid retrieval (BM25 + vector)
- e.g., Elasticsearch, MongoDB

Retrieval-Augmented Generation (RAG)

The primary mechanism for accessing long-term memory at inference time

Query

User input or agent question

→

Embed

Convert to vector representation

→

Retrieve

Search vector DB for top-K matches

→

Augment

Inject retrieved context into prompt

→

Generate

LLM produces grounded response

Pro Tip: Advanced RAG uses re-ranking, query decomposition, and iterative retrieval to improve recall and precision of long-term memory access.

03

Hybrid Memory Management

Combining short-term and long-term memory for optimal agent performance

Hybrid Memory Architecture

The hybrid approach uses a memory controller that dynamically routes between STM and LTM based on relevance, recency, and importance scores.

Short-Term Memory

Context window
Working buffer
Recent interactions

⟷

Memory Controller

Routes, scores, and manages memory flow

⟷

Long-Term Memory

Vector store
Knowledge graph
Document archive

Adaptive Retrieval

Dynamically selects memory source based on query type

Memory Consolidation

Important STM data is periodically promoted to LTM

Efficient Token Use

Only retrieves what's needed, preserving context budget

Memory Consolidation & Eviction

How memories move between tiers

1. **Score**
   Each memory gets importance, recency, and frequency scores
2. **Promote**
   High-score STM entries are written to LTM storage
3. **Compress**
   Similar memories are merged or summarized to save space
4. **Evict**
   Low-score entries are removed from STM to free tokens

Eviction Strategies

LRU (Least Recently Used)
Remove entries not accessed recently

LFU (Least Frequently Used)
Remove entries with fewest access counts

Importance-Based
Score by relevance; drop lowest-scored

TTL (Time-to-Live)
Auto-expire after a set duration

Semantic Dedup
Merge near-duplicate embeddings

04

User Memory

Personalizing agent behavior through stored user preferences, history, and profiles

User Memory: What Gets Stored

Preferences

- Communication style
- Preferred formats & tools
- Language and tone settings
- Notification preferences

Interaction History

- Past queries & tasks
- Feedback and corrections
- Frequently asked topics
- Session metadata

Profile & Context

- Role and domain expertise
- Active projects & goals
- Relationships & org context
- Skill level adaptations

Privacy, Consent & Ethical Considerations

Core Principles

Transparency
Users must know what is remembered and why

User Control
Ability to view, edit, and delete stored memories

Data Minimization
Store only what's necessary for the task at hand

Consent-Driven
Explicit opt-in before storing personal information

Risks to Mitigate

- ⚠ Memory poisoning attacks — adversarial data injected into memory
- ⚠ Stale preferences — outdated information leading to poor personalization
- ⚠ Over-personalization — creating filter bubbles or echo chambers
- ⚠ Cross-session leakage — information from one context bleeding into another
- ⚠ Compliance gaps — failure to meet GDPR, CCPA, or other regulations

Memory Types at a Glance

	Short-Term	Long-Term	Hybrid	User
Duration	Single session	Persistent	Dynamic	Persistent
Storage	Context window	Vector DB / KG	Both + controller	User profile DB
Access Speed	Instant	~100ms retrieval	Adaptive	Per-request
Capacity	4K–200K tokens	Millions of entries	Tiered	Per-user bounded
Update Freq.	Every turn	Batch / periodic	Event-driven	On user action
Key Benefit	Real-time context	Deep knowledge	Best of both	Personalization

05

Real-World Implementations

How leading AI companies and products implement memory systems

ChatGPT & Claude Code: Memory in Action

ChatGPT Memory

- **User Memory:** Persists preferences, facts, and instructions across sessions via explicit memory store
- **Context Window:** 128K token STM with GPT-4o; conversation history as working memory
- **Custom Instructions:** System-level persistent memory for tone, format, and role preferences
- **Retrieval (RAG):** GPTs use file search to query uploaded documents via vector embeddings
- **Memory Controls:** Users can view, edit, and delete individual memories; full transparency

Claude Code Memory

- **CLAUDE.md Files:** Project-level persistent memory storing codebase conventions, patterns, and preferences
- **Context Window:** 200K token STM; entire codebase context via smart file retrieval
- **Tool Use Memory:** Remembers tool outputs and file states within agentic coding sessions
- **User Memory:** Stores personal preferences, coding style, and workflow habits across projects
- **Hierarchical:** Global > project > session memory layers for multi-level personalization

Industry Memory Implementations

Google (Gemini)

1M+ token context window as massive STM; Google Search integration as LTM; user activity memory across Workspace

Microsoft (Copilot)

Microsoft Graph as LTM backbone; retrieves emails, docs, calendar for context; enterprise-grade user memory with M365 data

Meta (LLaMA)

Open-source approach; community-built memory layers; FAISS vector search for LTM; research on memory-augmented transformers

LangChain / LlamaIndex

Framework-level memory abstractions; ConversationBufferMemory, VectorStoreMemory, and hybrid chains for any LLM

Notion AI / Replit

Workspace-as-memory pattern; entire document/code history serves as retrievable LTM with semantic search

MemGPT / Letta

Virtual context management inspired by OS paging; automatic memory tiering between main context and external storage

Key Trend: The industry is converging on hybrid memory — combining large context windows with external retrieval and persistent user profiles.

MemGPT: LLMs as Operating Systems

Packer et al. (2023) — arXiv:2310.08560

The Key Insight

Treat LLM context like OS virtual memory — page data between fast main context and slow external storage.

Operating System		MemGPT
RAM	→	**Main Context**
Disk	→	**External Storage**
Page Faults	→	**Memory Pressure (70%)**
OS Kernel	→	**LLM as Memory Manager**

Two-Tier Architecture

Main Context (RAM): System prompt + core memory (bio, persona) + conversation FIFO queue + working context

External (Disk): Recall storage (searchable conversation history) + archival storage (unlimited vector DB)

Self-Directed: LLM decides when to read/write memory via function calls — no hard-coded rules

Memory Pressure: At 70% token usage, system warns the LLM to save important context before eviction

Key Takeaways

- Memory is what transforms stateless LLMs into persistent, context-aware agents
- Short-term memory provides real-time context but is limited by token budgets
- Long-term memory enables deep knowledge retention via vector DBs and knowledge graphs
- Hybrid systems combine both with intelligent routing for optimal performance
- User memory drives personalization while demanding strong privacy safeguards

Thank You

Agentic Memory Management

Agentic AI @ UIUC

Questions & Discussion